Sieci neuronowe – ćwiczenie 3

Nauka od zera: neuron, warstwa, sieć, propagacja wsteczna (część 1) oraz optymalizacja: paczki, ADAM, inicjalizacja He (część 2).

Jak czytać ten dokument: każdy wzór jest rozpisany linijka po linijce. Pod każdym wzorem jest ramka „co oznacza każdy symbol”, potem „po ludzku” i „na liczbach”. W przykładach zawsze są wypisane wszystkie dane, więc nie musisz niczego szukać w innych miejscach.

kolor ramki	co w niej jest
szara, kursywa	dosłowny cytat z treści listy
niebieska	wzór
przerywana	co oznacza każdy symbol we wzorze
żółta	po ludzku, bez wzorów
zielona	przykład na liczbach, każdy krok rachunku osobno
fioletowa	sprawdź się (pełne rozwiązania na końcu)

Plan nauki

etap	co	po czym poznasz, że umiesz
A. Symbole	co znaczy każda litera i znaczek	czytasz wzór $\frac{\partial L}{\partial w}$ na głos i wiesz, co mówisz
B. Macierze	mnożenie wiersz razy kolumna, transpozycja, $\odot$	liczysz Wx dla macierzy 2×2 na kartce
C. Pochodna	co to jest, 5 pochodnych, których potrzebujesz	mówisz, o ile zmieni się wynik, gdy wejście zmieni się o 0,001
1. Sieć	neuron, warstwa, ReLU, przejście w przód	liczysz wyjście sieci 2 → 2 → 1
2. Reguła łańcuchowa	mnożenie pochodnych ogniw, suma po drogach	liczysz $\frac{\partial L}{\partial w}$ dla jednego neuronu
3. Zapis wektorowy	jakobian, „idź w dół kolumny”	liczysz $\frac{\partial L}{\partial x}$ z tabelki
4. Element-wise	dlaczego dla ReLU wystarczy $\odot$	liczysz backward przez ReLU
5. forward / backward	moduły, kod z listy linijka po linijce	opowiadasz kod bez patrzenia
6. Warstwa liniowa	skąd $\frac{\partial L}{\partial W}$, $\frac{\partial L}{\partial b}$, $\frac{\partial L}{\partial x}$	wyprowadzasz te 3 wzory
7. Wyjście sieci	skąd $\delta = p - y$	wyprowadzasz to w 4 linijkach
8. Cała sieć	forward + backward + krok uczenia	robisz pełny przykład sam
9. Zadanie cz. 1	co badamy i co wyszło	tłumaczysz plateau i rolę normalizacji
10. Część 2	siodło, paczki, momentum, ADAM, He	liczysz krok ADAM i odchylenie He

11–13. Powtórka	ściąga, pytania, odpowiedzi	odpowiadasz na pytania bez notatek

A. Symbole: co znaczy każda litera

Najpierw słowniczek. Wróć tutaj zawsze, gdy we wzorze pojawi się coś niejasnego.

symbol	czytasz	co to jest
x	„iks”	wejście. Na początku sieci: cechy pacjenta (wiek, cholesterol, …). W środku sieci: **wejście jednego klocka** (patrz niżej).
y	„igrek”	**dwa znaczenia**, uwaga: (1) prawdziwa etykieta pacjenta: $y = 1$ chory, $y = 0$ zdrowy; (2) w treści listy o modułach: **wyjście jednego klocka**. Z kontekstu zawsze wiadomo, które.
w, W	„wu”	waga (jedna liczba) albo macierz wag (tabela liczb). Wagi to „pokrętła”, które sieć ustawia w trakcie uczenia.
b	„be”, bias	wyraz wolny: liczba dodawana na końcu, przesuwa wynik w górę lub w dół.
z	„zet”	suma ważona przed funkcją aktywacji: $z = w_1 x_1 + w_2 x_2 + b$.
a	„a”, aktywacja	wynik neuronu po funkcji aktywacji: $a = f(z)$.
p	„pe”	prawdopodobieństwo choroby, które podaje sieć, liczba od 0 do 1.
L	„el”, koszt (loss)	jedna liczba mówiąca, jak bardzo sieć się myli. Im mniejsza, tym lepiej.
α	„alfa”	współczynnik uczenia: jak długi krok robimy przy poprawianiu wag. W liście $\alpha = 0{,}01$.
e	„e”	stała $e \approx 2{,}718$.
$\ln$	„logarytm naturalny”	odwrotność $e^{\cdot}$: $\ln(e^3) = 3$. Ważne wartości: $\ln 1 = 0$, $\ln 0{,}5 = -0{,}693$, $\ln$ liczby z (0, 1) jest ujemny.
$\sigma(z)$	„sigma od zet”, sigmoida	$\sigma(z) = \frac{1}{1+e^{-z}}$, zamienia dowolną liczbę na liczbę z (0, 1). $\sigma(0) = 0{,}5$, $\sigma(1) = 0{,}731$, $\sigma(-1) = 0{,}269$.
$\sum_{j=1}^{n} x_j$	„suma po jot od 1 do en”	$x_1 + x_2 + \cdots + x_n$. Np. $\sum_{j=1}^{3} j = 1 + 2 + 3 = 6$.
$\frac{\partial L}{\partial w}$	„de el po de wu”	pochodna kosztu po wadze: o ile zmieni się L, gdy w zmienię o malutko. Szczegóły w rozdz. C.
$\odot$	„iloczyn element po elemencie”	$[1,2,3] \odot [4,5,6] = [1 \cdot 4,\ 2 \cdot 5,\ 3 \cdot 6] = [4,10,18]$.

x^T	„iks transponowane”	obrócenie: kolumna staje się wierszem, wiersz kolumną (rozdz. B).
$x \to y \to L$	„z iks liczymy igrek, z igrek liczymy el”	strzałka znaczy „**z tego liczymy to**”. To łańcuch obliczeń: x wpływa na L tylko przez y.
x_2, W_{12}	indeks dolny	numer elementu: x_2 to druga liczba w wektorze x; W_{12} to liczba w wierszu 1 i kolumnie 2 macierzy W.
W^1, W^2, a^1	indeks górny	**numer warstwy, a nie potęga!** W^1 to wagi warstwy 1, W^2 wagi warstwy 2.
$N(0;\ 0{,}01)$	„rozkład normalny”	losowanie z rozkładu Gaussa o średniej 0 i odchyleniu standardowym 0,01, czyli liczby typu 0,004, −0,012, 0,009.

A.1. „Klocek”, czyli moduł

Sieć liczy wynik w kilku krokach. Każdy krok to **klocek** (moduł): bierze coś na wejściu (x) i oddaje coś na wyjściu (y). Wyjście jednego klocka jest wejściem następnego:

$$\text{cechy} \ \to\ \boxed{Wx+b}\ \to\ \boxed{\text{ReLU}}\ \to\ \boxed{Wx+b}\ \to\ \boxed{\text{sigmoida}}\ \to\ p\ \to\ \boxed{\text{koszt}}\ \to\ L$$

Po ludzku: dla klocka ReLU „x” to jego wejście (to, co oddał poprzedni klocek), a „y” to jego wyjście. To **nie** są cechy pacjenta ani etykieta. Dlatego w każdym klocku x i y znaczą coś innego.

B. Wektory i macierze na liczbach

B.1. Wektor i macierz

Wektor to lista liczb. Może być zapisany w pionie (kolumna) albo w poziomie (wiersz):

$$x = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \text{ (kolumna, kształt } (2,1)), \qquad x^T = \begin{bmatrix} 1 & -1 \end{bmatrix} \text{ (wiersz, kształt } (1,2))$$

Macierz to tabela liczb. Kształt zapisujemy jako (liczba wierszy, liczba kolumn):

$$W = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix} \quad \text{kształt } (2,2), \qquad W_{11}=2,\ W_{12}=1,\ W_{21}=-1,\ W_{22}=3$$

B.2. Mnożenie macierzy przez wektor: Wx

Każdy **wiersz** W mnożysz element po elemencie przez wektor x i sumujesz. Każdy wiersz daje jedną liczbę wyniku.

Na liczbach: $W = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $x = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

1. wiersz 1 to $[2,\ 1]$: $2 \cdot 1 + 1 \cdot (-1) = 2 - 1 = 1$
2. wiersz 2 to $[-1,\ 3]$: $(-1) \cdot 1 + 3 \cdot (-1) = -1 - 3 = -4$
3. wynik: $Wx = \begin{bmatrix} 1 \\ -4 \end{bmatrix}$

Kształty muszą pasować: $(2, \mathbf{2}) \cdot (\mathbf{2}, 1) = (2,1)$. Środkowe liczby muszą być równe (tyle kolumn w W, ile elementów w x), a wynik ma kształt z liczb zewnętrznych. Dlatego „kolejność ma znaczenie”: Wx działa, a xW dla kolumny x nie.

B.3. Wiersz razy macierz: vW

Na liczbach: $v = [3,\ 1]$ (wiersz), $W = \begin{bmatrix} 2 & 1 \\ 0 & 4 \end{bmatrix}$. Teraz wiersz v mnożysz przez każdą **kolumnę** W:

1. kolumna 1 to $[2,\ 0]$: $3 \cdot 2 + 1 \cdot 0 = 6$
2. kolumna 2 to $[1,\ 4]$: $3 \cdot 1 + 1 \cdot 4 = 7$
3. wynik: $vW = [6,\ 7]$, kształt $(1,2) \cdot (2,2) = (1,2)$

B.4. Transpozycja W^T

Wiersze stają się kolumnami: element z wiersza i, kolumny j przechodzi do wiersza j, kolumny i.

$$W = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix} \quad \Rightarrow \quad W^T = \begin{bmatrix} 2 & -1 \\ 1 & 3 \end{bmatrix}, \qquad W = [1{,}5\ \ 2] \ \Rightarrow\ W^T = \begin{bmatrix} 1{,}5 \\ 2 \end{bmatrix}$$

B.5. Kolumna razy wiersz (iloczyn zewnętrzny)

Kolumna $(n,1)$ razy wiersz $(1,m)$ daje macierz (n,m): w kratce (i,j) stoi i-ty element kolumny razy j-ty element wiersza.

Na liczbach:

$$\begin{bmatrix} 0{,}5 \\ -1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0{,}5 \cdot 1 & 0{,}5 \cdot 2 \\ (-1) \cdot 1 & (-1) \cdot 2 \end{bmatrix} = \begin{bmatrix} 0{,}5 & 1 \\ -1 & -2 \end{bmatrix}$$

Tego użyjemy do liczenia gradientu wag (rozdz. 6).

B.6. Iloczyn element po elemencie $\odot$

Dwa wektory tej samej długości: mnożysz pierwszy z pierwszym, drugi z drugim itd. Nic się nie sumuje.

$$[4,\ 2,\ 5] \odot [0,\ 1,\ 1] = [4 \cdot 0,\ 2 \cdot 1,\ 5 \cdot 1] = [0,\ 2,\ 5]$$

Sprawdź się B: (a) Policz Wx dla $W = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $x = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$. (b) Jaki kształt ma wynik $(8, 17) \cdot (17, 1)$? (c) Policz $\begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 3 & -1 \end{bmatrix}$.

C. Pochodna: co to jest i których potrzebujesz

C.1. Definicja na jednej liczbie

$$f'(x) = \frac{df}{dx} \approx \frac{f(x+h) - f(x)}{h} \quad \text{dla bardzo małego } h$$

Co oznacza każdy symbol: $f(x)$ – funkcja, wynik dla wejścia x; h – malutka zmiana wejścia (np. 0,001); $f(x+h) - f(x)$ – o ile zmienił się wynik; dzielenie przez h – zmiana wyniku **na jednostkę** zmiany wejścia.

Po ludzku: pochodna mówi, **ile razy mocniej** zmienia się wynik niż wejście. Pochodna 6 znaczy: ruszysz wejściem o 0,001, a wynik ruszy się o ok. 0,006. Znak mówi o kierunku: dodatnia – wynik rośnie razem z wejściem, ujemna – maleje.

Na liczbach: $f(x) = x^2$, $x = 3$, $h = 0{,}001$.

1. $f(3) = 9$
2. $f(3{,}001) = 3{,}001^2 = 9{,}006001$
3. zmiana wyniku: $9{,}006001 - 9 = 0{,}006001$
4. na jednostkę: $0{,}006001 / 0{,}001 = 6{,}001 \approx 6$
5. wzór $f'(x) = 2x$ daje $2 \cdot 3 = 6$ ✓

C.2. Pochodna cząstkowa ∂

Gdy funkcja ma kilka wejść, $\frac{\partial f}{\partial x_1}$ znaczy: ruszam **tylko** x_1, resztę traktuję jak stałe liczby.

Na liczbach: $f(x_1, x_2) = 2x_1 + x_1 x_2$.

1. $\frac{\partial f}{\partial x_1}$: x_2 to stała, więc $2x_1 \to 2$, a $x_1 x_2 \to x_2$. Wynik: $2 + x_2$.
2. $\frac{\partial f}{\partial x_2}$: x_1 to stała, więc $2x_1 \to 0$, a $x_1 x_2 \to x_1$. Wynik: x_1.
3. dla $x_1 = 3$, $x_2 = 5$: $\frac{\partial f}{\partial x_1} = 7$, $\frac{\partial f}{\partial x_2} = 3$.

C.3. Pięć pochodnych, których potrzebujesz

funkcja	pochodna	przykład
$f(x) = c \cdot x$ (c – stała)	c	$f = 3x \Rightarrow f' = 3$
$f(x) = x^2$	$2x$	dla $x = 3$: 6
$f(x) = \ln x$	$\frac{1}{x}$	dla $x = 0{,}5$: 2
$f(z) = \sigma(z)$	$\sigma(z)\,(1 - \sigma(z))$	dla $z = 1$: $0{,}731 \cdot 0{,}269 = 0{,}197$
$f(z) = \mathrm{ReLU}(z) = \max(0, z)$	1 gdy $z > 0$, 0 gdy $z \leq 0$	dla $z = 3$: 1; dla $z = -2$: 0

C.4. Skąd pochodna sigmoidy $\sigma(1 - \sigma)$

1. $\sigma(z) = \frac{1}{1 + e^{-z}} = (1 + e^{-z})^{-1}$

2. pochodna $(\dots)^{-1}$ to $-(\dots)^{-2}$ razy pochodna środka; pochodna środka $1+e^{-z}$ to $-e^{-z}$: $\sigma'(z) = -(1+e^{-z})^{-2}\cdot(-e^{-z}) = \dfrac{e^{-z}}{(1+e^{-z})^2}$

3. rozbijam na iloczyn: $\dfrac{e^{-z}}{(1+e^{-z})^2} = \dfrac{1}{1+e^{-z}}\cdot\dfrac{e^{-z}}{1+e^{-z}}$

4. pierwszy czynnik to $\sigma(z)$; drugi: $\dfrac{e^{-z}}{1+e^{-z}} = \dfrac{(1+e^{-z})-1}{1+e^{-z}} = 1-\dfrac{1}{1+e^{-z}} = 1-\sigma(z)$

5. razem: $\sigma'(z) = \sigma(z)\,(1-\sigma(z))$

Sprawdź się C: (a) $f(x)=x^2$, $x=-2$. Ile wynosi $f'(-2)$ i o ile mniej więcej zmieni się f, gdy x zmienię o $+0{,}01$? (b) $f(x_1,x_2)=x_1^2+4x_2$. Policz $\frac{\partial f}{\partial x_1}$ i $\frac{\partial f}{\partial x_2}$ dla $x_1=1$, $x_2=7$.

1. Neuron, warstwa, sieć

1.1. Neuron

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b, \qquad a = f(z)$$

Co oznacza każdy symbol: $x_1, \ldots, x_n$ – liczby na wejściu neuronu (np. cechy pacjenta); $w_1, \ldots, w_n$ – wagi neuronu, po jednej na każde wejście; b – bias; z – suma ważona; f – funkcja aktywacji; a – wynik neuronu (jedna liczba).

Po ludzku: neuron to kalkulator: mnoży każde wejście przez jego wagę, dodaje wszystko i bias, a na końcu przepuszcza przez funkcję. Duża dodatnia waga = to wejście mocno podnosi wynik. Ujemna = obniża.

Na liczbach: wejścia $x = [1,\ 2]$, wagi $w = [0{,}5,\ -1]$, bias $b = 1$, funkcja: sigmoida.

1. $z = 0{,}5 \cdot 1 + (-1) \cdot 2 + 1 = 0{,}5 - 2 + 1 = -0{,}5$
2. $a = \sigma(-0{,}5) = \dfrac{1}{1 + e^{0{,}5}} = \dfrac{1}{1 + 1{,}649} = 0{,}378$
3. neuron mówi: 37,8% szans na chorobę.

Jeden neuron z sigmoidą to dokładnie **regresja logistyczna**.

1.2. Warstwa = kilka neuronów, które liczą z tych samych wejść

Wagi wszystkich neuronów warstwy zapisujemy jako **wiersze** jednej macierzy W. Wtedy cała warstwa to jedno mnożenie:

$$z = Wx + b, \qquad a = f(z)\ \text{(osobno dla każdego elementu)}$$

Co oznacza każdy symbol: x – kolumna wejść, kształt $(d_{in}, 1)$; W – macierz wag, kształt (d_{out}, d_{in}), wiersz i = wagi neuronu i; b – kolumna biasów, po jednym na neuron, kształt $(d_{out}, 1)$; z, a – kolumny wyników, po jednej liczbie na neuron. d_{in} – liczba wejść, d_{out} – liczba neuronów.

Po ludzku: każdy neuron w warstwie uczy się reagować na inną kombinację cech, np. „starszy i wysoki cholesterol”. Warstwa z 17 neuronami zamienia pacjenta na 17 nowych liczb.

1.3. Funkcja aktywacji i ReLU

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{gdy } z > 0 \\ 0 & \text{gdy } z \le 0 \end{cases}$$

Po ludzku: ujemne zamieniasz na 0, resztę zostawiasz. $\mathrm{ReLU}(5) = 5$, $\mathrm{ReLU}(0) = 0$, $\mathrm{ReLU}(-4) = 0$. Neuron, który dał 0, jest „wyłączony”.

Po co funkcja aktywacji? Bez niej dwie warstwy to dalej jedna warstwa liniowa:

1. warstwa 1: $z^1 = W^1 x$, warstwa 2: $z^2 = W^2 z^1$
2. podstawiam: $z^2 = W^2(W^1 x) = (W^2 W^1)\, x$
3. $W^2 W^1$ to jedna macierz, więc całość to jedna warstwa. Na liczbach: „razy 3”, potem „razy 2” to to samo co „razy 6”.

ReLU między warstwami „zgina” wynik i tego zwinięcia już nie da się zrobić, więc sieć może narysować krzywą granicę między chorymi a zdrowymi.

funkcja	wzór	pochodna	wyniki z przedziału
sigmoida	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	(0, 1)
ReLU	$\max(0, z)$	1 dla $z > 0$, 0 dla $z \leq 0$	[0, ∞)
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$	(−1, 1)

1.4. Sieć (MLP) i przejście w przód

MLP (Multilayer Perceptron) to warstwy jedna za drugą. Warstwy w środku to **warstwy ukryte**. W liście: warstwy ukryte z ReLU, na końcu jeden neuron z sigmoidą (prawdopodobieństwo choroby), koszt to entropia krzyżowa.

$$z^1 = W^1 x + b^1, \quad a^1 = \mathrm{ReLU}(z^1), \quad z^2 = W^2 a^1 + b^2, \quad p = \sigma(z^2)$$

Co oznacza każdy symbol: górny indeks to numer warstwy; a^1 – wyniki warstwy ukrytej, stają się wejściem warstwy 2; p – wynik sieci.

Na liczbach: sieć 2 → 2 → 1 (2 cechy, 2 neurony ukryte, 1 wyjście).
Dane: pacjent $x = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ (wiek 1 odchylenie powyżej średniej, cholesterol 1 poniżej), $W^1 = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $b^1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $W^2 = [1{,}5 \;\; 2]$, $b^2 = -0{,}5$.

1. neuron 1, wiersz $[2,\ 1]$: $z_1^1 = 2 \cdot 1 + 1 \cdot (-1) + 0 = 1$
2. neuron 2, wiersz $[-1,\ 3]$: $z_2^1 = (-1) \cdot 1 + 3 \cdot (-1) + 0 = -4$
3. ReLU: $a^1 = [\mathrm{ReLU}(1),\ \mathrm{ReLU}(-4)] = [1,\ 0]$ (neuron 2 wyłączony)
4. wyjście: $z^2 = 1{,}5 \cdot 1 + 2 \cdot 0 - 0{,}5 = 1$
5. $p = \sigma(1) = \frac{1}{1+e^{-1}} = \frac{1}{1+0{,}368} = 0{,}731$: sieć mówi 73,1% szans na chorobę.

Sprawdź się 1: Ta sama sieć ($W^1 = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $b^1 = [0, 0]$, $W^2 = [1{,}5 \;\; 2]$, $b^2 = -0{,}5$), pacjent $x = [-1,\ 2]$. Policz z^1, a^1, z^2, p.

2. Reguła łańcuchowa

> *„Ćwiczenie 3 obejmuje algorytm propagacji wstecznej (...) Propagacja wsteczna jest po prostu zastosowaniem reguły łańcuchowej: ∂L/∂x = ∂L/∂y · ∂y/∂x, w wyliczaniu gradientu kosztu modelu po parametrach.”*

2.1. Po co nam pochodne

Sieć uczy się tak: każdą wagę przesuwamy trochę w stronę, w której koszt maleje.

$$w_{\text{nowe}} = w_{\text{stare}} - \alpha \cdot \frac{\partial L}{\partial w}$$

Co oznacza każdy symbol: $\frac{\partial L}{\partial w}$ – o ile rośnie koszt, gdy waga rośnie; α – długość kroku; minus – idziemy w przeciwną stronę niż wzrost kosztu.

Na liczbach: $w = 2$, $\frac{\partial L}{\partial w} = 0{,}5$ (koszt rośnie, gdy waga rośnie), $\alpha = 0{,}1$: $w_{\text{nowe}} = 2 - 0{,}1 \cdot 0{,}5 = 1{,}95$. Waga zmalała, więc koszt zmaleje.

Problem: koszt nie zależy od wag bezpośrednio, tylko przez cały łańcuch obliczeń. Reguła łańcuchowa mówi, jak policzyć pochodną takiego łańcucha.

2.2. Reguła łańcuchowa dla jednej drogi

Jeśli $x \to y \to L$ (z x liczymy y, z y liczymy L), to

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Dłuższy łańcuch $x \to y \to z \to L$: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$ – mnożysz pochodne wszystkich ogniw.

Co oznacza każdy symbol: $\frac{\partial y}{\partial x}$ – jak mocno y reaguje na x (jedno ogniwo); $\frac{\partial L}{\partial y}$ – jak mocno L reaguje na y (następne ogniwo); iloczyn – jak mocno L reaguje na x przez cały łańcuch.

Po ludzku: jeśli y zmienia się 3 razy mocniej niż x, a L 2 razy mocniej niż y, to L zmienia się $3 \cdot 2 = 6$ razy mocniej niż x.

Na liczbach, najprostszy łańcuch: $y = 3x$, $L = y^2$, $x = 1$.

1. w przód: $y = 3 \cdot 1 = 3$, $L = 3^2 = 9$
2. ogniwo $y \to L$: $L = y^2$, pochodna $\frac{\partial L}{\partial y} = 2y = 2 \cdot 3 = 6$
3. ogniwo $x \to y$: $y = 3x$, pochodna $\frac{\partial y}{\partial x} = 3$
4. łańcuch: $\frac{\partial L}{\partial x} = 6 \cdot 3 = 18$
5. sprawdzenie: $L = (3x)^2 = 9x^2$, więc $\frac{dL}{dx} = 18x = 18$ ✓

2.3. Reguła łańcuchowa dla jednego neuronu z sigmoidą

Łańcuch: $w \to z \to p \to L$, gdzie $z = w \cdot x$, $p = \sigma(z)$, a koszt dla pacjenta zdrowego ($y = 0$) to $L = -\ln(1-p)$.

Na liczbach: wejście $x = 2$, waga $w = 0{,}5$, pacjent zdrowy, $y = 0$.
W przód:

1. $z = w \cdot x = 0{,}5 \cdot 2 = 1$
2. $p = \sigma(1) = 0{,}731$
3. $L = -\ln(1 - 0{,}731) = -\ln(0{,}269) = 1{,}313$

Wstecz, ogniwo po ogniwie:

4. ogniwo $p \to L$: $L = -\ln(1-p)$. Pochodna $\ln(u)$ to $\frac{1}{u}$, a $u = 1-p$ maleje, gdy p rośnie (pochodna -1). Razem z minusem przed $\ln$: $\frac{\partial L}{\partial p} = -\frac{1}{1-p} \cdot (-1) = \frac{1}{1-p} = \frac{1}{0{,}269} = 3{,}718$
5. ogniwo $z \to p$: pochodna sigmoidy, $\frac{\partial p}{\partial z} = p(1-p) = 0{,}731 \cdot 0{,}269 = 0{,}197$
6. ogniwo $w \to z$: $z = w \cdot x$, więc $\frac{\partial z}{\partial w} = x = 2$
7. łańcuch: $\frac{\partial L}{\partial w} = 3{,}718 \cdot 0{,}197 \cdot 2 = 1{,}462$
8. skrót: $(p - y) \cdot x = (0{,}731 - 0) \cdot 2 = 1{,}462$ ✓ – to samo!

Dlaczego skrót działa: w krokach 4 i 5 mnożymy $\frac{1}{1-p} \cdot p(1-p)$, a $(1-p)$ się skraca i zostaje samo $p = p - 0 = p - y$. Ogólny dowód jest w rozdz. 7.

2.4. Kilka dróg = suma

Jeśli x wpływa na L dwiema drogami, przez y_1 i przez y_2:

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial y_1} \cdot \frac{\partial y_1}{\partial x}}_{\text{droga przez } y_1} + \underbrace{\frac{\partial L}{\partial y_2} \cdot \frac{\partial y_2}{\partial x}}_{\text{droga przez } y_2}$$

Po ludzku: na każdej drodze mnożysz (jak wyżej), a potem drogi dodajesz, bo x zmienia L jednocześnie przez obie. W sieci każde wejście warstwy idzie do każdego neuronu, więc dróg jest dużo – dlatego potrzebny jest porządny zapis z rozdz. 3.

Na liczbach: $y_1 = 2x_1 + x_2$, $y_2 = 4x_2$, a z dalszej części sieci wiemy: $\frac{\partial L}{\partial y_1} = 3$, $\frac{\partial L}{\partial y_2} = 1$. Szukam $\frac{\partial L}{\partial x_2}$.

1. droga przez y_1: $\frac{\partial y_1}{\partial x_2} = 1$ (przy x_2 stoi 1), więc $3 \cdot 1 = 3$
2. droga przez y_2: $\frac{\partial y_2}{\partial x_2} = 4$, więc $1 \cdot 4 = 4$
3. suma: $\frac{\partial L}{\partial x_2} = 3 + 4 = 7$
4. dla x_1 jest jedna droga (przez y_1, bo w y_2 nie ma x_1): $3 \cdot 2 = 6$

2.5. Co to jest propagacja wsteczna

W sieci łańcuch jest długi: $W^1 \to z^1 \to a^1 \to z^2 \to p \to L$. Propagacja wsteczna liczy pochodne **od końca**: najpierw $\frac{\partial L}{\partial p}$, z niego $\frac{\partial L}{\partial z^2}$, z niego $\frac{\partial L}{\partial a^1}$ itd. Każdy wynik pośredni jest od razu używany do

następnego ogniwa, więc nic nie liczymy dwa razy.

Sprawdź się 2: Łańcuch $w \to z \to L$, gdzie $z = 3w$, $L = z^2$, $w = 1$. Policz z, L, $\frac{\partial L}{\partial z}$, $\frac{\partial z}{\partial w}$, $\frac{\partial L}{\partial w}$.

3. Zapis wektorowy i kształty

> *„Operujemy na wektorach i mnożeniu macierzowym, kolejność ma znaczenie. L jest skalarem, w notacji gdzie x to wektor kolumnowy, ∂L/∂x jest wektorem o wymiarze odpowiadającym x^T (aby zgadzało się mnożenie macierzowe). ∂y/∂x to macierz pochodnych cząstkowych wszystkich wyjść po wszystkich wejściach o wymiarach (dim_y, dim_x).”*

3.1. Każde zdanie cytatu osobno

zdanie z listy	co znaczy
„L jest skalarem”	koszt to jedna liczba (skalar = pojedyncza liczba, nie wektor)
„x to wektor kolumnowy”	wejście klocka zapisujemy pionowo, kształt $(\dim_x, 1)$, np. $\begin{bmatrix}1\\-1\end{bmatrix}$
„∂L/∂x ma wymiar jak x^T”	gradient zapisujemy poziomo, jako **wiersz** $(1, \dim_x)$: jedna pochodna na każdy element x. Np. $\frac{\partial L}{\partial x} = [6,\ 7]$ znaczy $\frac{\partial L}{\partial x_1} = 6$, $\frac{\partial L}{\partial x_2} = 7$
„∂y/∂x to macierz (dim_y, dim_x)”	tabela wszystkich pochodnych: **wiersz i = wyjście y_i**, **kolumna j = wejście x_j**, w kratce $\frac{\partial y_i}{\partial x_j}$. Nazywa się **jakobian**
„kolejność ma znaczenie”	wiersz $\frac{\partial L}{\partial y}$ mnożymy przez jakobian, w tej kolejności: $(1, \dim_y) \cdot (\dim_y, \dim_x) = (1, \dim_x)$ ✓

$$\underbrace{\frac{\partial L}{\partial x}}_{(1,\,\dim_x)} = \underbrace{\frac{\partial L}{\partial y}}_{(1,\,\dim_y)} \cdot \underbrace{\frac{\partial y}{\partial x}}_{(\dim_y,\,\dim_x)}$$

To jest reguła łańcuchowa z rozdz. 2 zapisana dla wszystkich dróg naraz. Mnożenie wiersza przez macierz robi dokładnie „pomnóż na każdej drodze i zsumuj drogi”.

3.2. Przepis „tabelka”

1. Zrób tabelkę: **wiersz = które** y, **kolumna = po którym** x, w kratce $\frac{\partial y_i}{\partial x_j}$.
2. Obok wierszy dopisz liczby $\frac{\partial L}{\partial y_i}$, które przyszły „z góry” (od klocka dalej w sieci).
3. Dla każdego x_j **idź w dół jego kolumny**: każdą kratkę mnożysz przez $\frac{\partial L}{\partial y_i}$ z tego samego wiersza i sumujesz.

Na liczbach: klocek $y_1 = 2x_1 + x_2,\ \ y_2 = 4x_2$; z góry $\frac{\partial L}{\partial y} = [3,\ 1]$.

	po x_1	po x_2	z góry $\frac{\partial L}{\partial y_i}$
$y_1 = 2x_1 + x_2$	2	1	3
$y_2 = 4x_2$	0	4	1

1. kolumna x_1: $2 \cdot 3 + 0 \cdot 1 = 6$
2. kolumna x_2: $1 \cdot 3 + 4 \cdot 1 = 7$
3. $\frac{\partial L}{\partial x} = [6,\ 7]$ – to samo co w rozdz. 2.4 liczone drogami ✓
4. macierzowo: $[3,\ 1] \cdot \begin{bmatrix}2 & 1\\0 & 4\end{bmatrix} = [6,\ 7]$ (rachunek w rozdz. B.3)

Po ludzku: klocek nie musi znać wzoru na L. Dostaje gotowe $\frac{\partial L}{\partial y}$ „z góry” i oddaje $\frac{\partial L}{\partial x}$ „w dół”. Dla klocka niżej to, co oddałeś, jest jego „z góry”.

Sprawdź się 3: (a) Warstwa ma 17 wejść i 8 wyjść. Jaki kształt mają x, jakobian $\frac{\partial y}{\partial x}$ i $\frac{\partial L}{\partial x}$? (b) Klocek: $y_1 = x_1 + 2x_3$, $y_2 = 3x_2 - x_3$, z góry $\frac{\partial L}{\partial y} = [2,\ 5]$. Zrób tabelkę 2×3 i policz $\frac{\partial L}{\partial x}$.

4. Funkcje element-wise i backward przez ReLU

> *„Weźmy przykład dowolnej funkcji element-wise, tzn. takiej która aplikuje przekształcenie niezależnie od siebie do każdego elementu wektora np. f(x) = x² (...) Dla wektorów powinniśmy mieć jako ∂y/∂x macierz kwadratową (dim_x, dim_x) i wykorzystać ją w regule łańcuchowej, ale zauważmy, że wszystkie elementy tej macierzy poza diagonalą będą zerowe, natomiast diagonala to po prostu elementy wektora 2x."*

4.1. Najpierw dla jednej liczby

1. $f(x) = x^2$, pochodna $f'(x) = 2x$
2. jeśli dalej jest funkcja g, czyli $h(x) = g(f(x)) = g(x^2)$, to z reguły łańcuchowej $h'(x) = g'(x^2) \cdot 2x$ (pochodna zewnętrznej, policzona w punkcie x^2, razy pochodna wewnętrznej)

4.2. Dla wektora: dlaczego jakobian jest diagonalny

„Element-wise" znaczy: $y_1 = x_1^2$, $y_2 = x_2^2$, $y_3 = x_3^2$. Każde y_i zależy **tylko od swojego** x_i.

1. $\frac{\partial y_1}{\partial x_1} = 2x_1$ (to samo co dla liczby)
2. $\frac{\partial y_1}{\partial x_2} = 0$, bo w $y_1 = x_1^2$ nie ma x_2; tak samo każda kratka poza przekątną
3. jakobian: $\frac{\partial y}{\partial x} = \begin{bmatrix} 2x_1 & 0 & 0 \\ 0 & 2x_2 & 0 \\ 0 & 0 & 2x_3 \end{bmatrix}$ – na przekątnej (diagonali) elementy wektora $2x$, reszta zera

Na liczbach: $x = [1,\ 2,\ 3]$, z góry $\frac{\partial L}{\partial y} = [0{,}5,\ 1,\ 2]$.

1. przekątna $2x = [2,\ 4,\ 6]$
2. pełne mnożenie wiersza przez macierz: $[0{,}5,\ 1,\ 2] \cdot \begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 6 \end{bmatrix}$; kolumna 1: $0{,}5 \cdot 2 + 1 \cdot 0 + 2 \cdot 0 = 1$; kolumna 2: $0 + 1 \cdot 4 + 0 = 4$; kolumna 3: $0 + 0 + 2 \cdot 6 = 12$. Wynik $[1,\ 4,\ 12]$
3. element po elemencie: $[0{,}5,\ 1,\ 2] \odot [2,\ 4,\ 6] = [1,\ 4,\ 12]$ – **to samo**, a zero mnożeń przez zera

Dla każdej funkcji element-wise $y = f(x)$:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \odot f'(x)$$

Co oznacza każdy symbol: $\frac{\partial L}{\partial y}$ – wiersz, który przyszedł z góry; $f'(x)$ – wektor pochodnych policzonych w każdym elemencie wejścia; $\odot$ – mnożenie element po elemencie (rozdz. B.6).

Wszystkie funkcje aktywacji (ReLU, sigmoida, tanh) są element-wise, więc ich backward zawsze tak wygląda. To jest zdanie z listy: „przemnożenie przez macierz diagonalną jest zastąpione mnożeniem element-wise".

4.3. Backward przez ReLU

Pochodna ReLU to 1 tam, gdzie wejście było dodatnie, i 0 w pozostałych miejscach. Taki wektor zer i jedynek to **maska**, piszemy ją $[z > 0]$.

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \odot [z > 0]$$

Na liczbach: wejście ReLU $z = [-2,\ 3,\ 1]$, z góry $\frac{\partial L}{\partial a} = [4,\ 2,\ 5]$.

1. w przód: $a = [\max(0,-2),\ \max(0,3),\ \max(0,1)] = [0,\ 3,\ 1]$
2. maska: $-2 > 0$? nie → 0; $3 > 0$? tak → 1; $1 > 0$? tak → 1. Maska $= [0,\ 1,\ 1]$
3. wstecz: $[4,\ 2,\ 5] \odot [0,\ 1,\ 1] = [0,\ 2,\ 5]$

Po ludzku: ReLU przepuszcza gradient tam, gdzie neuron był włączony, i blokuje (daje 0) tam, gdzie był wyłączony. Neuron, który w przód dał 0, wstecz nic nie przekaże i jego wagi w tym kroku się nie zmienią.

Sprawdź się 4: Wejście ReLU $z = [-1,\ 2,\ 0{,}5]$, z góry $\frac{\partial L}{\partial a} = [3,\ 1,\ 4]$. Policz maskę i $\frac{\partial L}{\partial z}$.

5. forward i backward: moduły

> *„Pamiętajmy też że wyliczanie odpowiednich wartości wyjściowych z każdej operacji musi odbyć się w kolejności tych operacji, natomiast wyliczanie gradientu po konkretnych wejściach – w kolejności odwrotnej.”*

5.1. Kod z listy, linijka po linijce

```
// x.shape=(x,1), żeby zgadzać się z opisaną wyżej konwencją
// zapisu. W backward() potrzebna będzie transpozycja

def forward(x):
     cache_x = x
     return x*x

// derivative_y.shape=(1,x)
def backward(derivative_y):
     return derivative_y*2*cache_x.transpose()
```

linijka	co robi i dlaczego
`x.shape=(x,1)`	wejście to kolumna (rozdz. 3)
`def forward(x):`	przejście w przód tego klocka
`cache_x = x`	zapamiętuje wejście. Pochodna x^2 to $2x$, czyli w backward będzie potrzebne x, a wtedy nikt go już nie poda
`return x*x`	wyjście $y = x^2$ (element po elemencie)
`derivative_y.shape=(1,x)`	$\frac{\partial L}{\partial y}$, które przychodzi z góry, to wiersz
`def backward(derivative_y):`	przejście wstecz: dostaje $\frac{\partial L}{\partial y}$
`derivative_y*2*cache_x`	$\frac{\partial L}{\partial y} \odot 2x$, czyli wzór z rozdz. 4.2
`.transpose()`	`cache_x` to kolumna, a `derivative_y` wiersz. Żeby mnożyć element po elemencie, oba muszą mieć ten sam kształt, więc kolumnę obracamy w wiersz
`return …`	oddaje $\frac{\partial L}{\partial x}$ (wiersz) klockowi poniżej

Na liczbach: forward z $x = \begin{bmatrix}1\\2\\3\end{bmatrix}$: `cache_x` $= \begin{bmatrix}1\\2\\3\end{bmatrix}$, zwraca $\begin{bmatrix}1\\4\\9\end{bmatrix}$. Backward z `derivative_y` $= [0{,}5,\ 1,\ 2]$: `cache_x.transpose()` $= [1,\ 2,\ 3]$; zwraca $[0{,}5 \cdot 2 \cdot 1,\ 1 \cdot 2 \cdot 2,\ 2 \cdot 2 \cdot 3] = [1,\ 4,\ 12]$.

5.2. Sieć = ciąg modułów

> *„Wielowarstwowa sieć neuronowa uczona algorytmem propagacji wstecznej to nic innego jak sekwencja modułów obsługujących forward i backward. Na potrzeby prostej sieci MLP potrzebujemy: przekształcenia liniowego Wx+b, funkcji aktywacji, funkcji kosztu.”*

```
forward:   x  → Linear1 → ReLU → Linear2 → Sigmoida → Koszt → L
backward: ∂L/∂x ← Linear1 ← ReLU ← Linear2 ← Sigmoida ← Koszt ← start
```

5.3. Cztery zasady z listy

z listy	w praktyce
„Przy przejściu w przód, w pamięci muszą pozostać wartości wektorów wejściowych do każdej operacji”	każdy moduł w forward zapisuje swoje wejście (cache), bo backward go potrzebuje
„backward na wejściu przyjmuje wyliczone do tej pory ∂L/∂y, zaś na wyjściu daje ∂L/∂x”	to, co oddajesz, jest „z góry” dla modułu niżej
„backward nie zawsze musi być pełnym mnożeniem macierzy ∂y/∂x”	dla funkcji aktywacji wystarczy ⊙ (rozdz. 4)
„jeżeli korzystamy z wag modelu, powinniśmy wyliczyć i zapisać również gradient ∂L/∂W_i a następnie zwrócić ∂L/∂y_{i-1}”	warstwa liniowa w backward liczy dwie rzeczy: $\frac{\partial L}{\partial W}$ i $\frac{\partial L}{\partial b}$ zostawia u siebie (do kroku uczenia), a $\frac{\partial L}{\partial x}$ oddaje w dół

Po ludzku: jeśli każdy klocek ma forward i backward, to można je układać w dowolnej liczbie. Sieć z 1, 2 czy 5 warstwami to tylko dłuższa lista klocków, a kod uczenia się nie zmienia.

5.4. Jak to wygląda w Twoim kodzie (mlp.py)

```
class Linear:
    def forward(self, X):
        self.X = X                      # cache wejścia
        return X @ self.W.T + self.b    # z = Wx + b dla wszystkich pacjentów naraz
    def backward(self, dY):             # dY = ∂L/∂z, kształt (N, n_out)
        self.dW = dY.T @ self.X         # ∂L/∂W  (zostaje, do kroku uczenia)
        self.db = dY.sum(0)             # ∂L/∂b
        return dY @ self.W              # ∂L/∂x  (idzie w dół)

class ReLU:
    def forward(self, X):
        self.mask = X > 0               # cache: maska [z > 0]
        return X * self.mask
    def backward(self, dY):
        return dY * self.mask           # ∂L/∂a ⊙ [z > 0]
```

W kodzie każdy **wiersz** macierzy X to jeden pacjent, więc X ma kształt (N, cechy), a `X @ W.T` liczy Wx dla wszystkich N pacjentów naraz. Wzory są te same, tylko obrócone (transpozycja).

Sprawdź się 5: (a) Dlaczego forward musi zapamiętać wejście? (b) Co backward warstwy liniowej oddaje w dół, a co zatrzymuje u siebie?

6. Warstwa liniowa Wx + b: skąd biorą się gradienty

Forward: $z = Wx + b$. Backward dostaje z góry $\delta = \frac{\partial L}{\partial z}$ („delta”, błąd warstwy, po jednej liczbie na neuron). Ma policzyć trzy rzeczy:

$$\frac{\partial L}{\partial W} = \delta \cdot x^T, \qquad \frac{\partial L}{\partial b} = \delta, \qquad \frac{\partial L}{\partial x} = W^T \delta$$

Co oznacza każdy symbol: δ – kolumna $(d_{out}, 1)$, błąd każdego neuronu; x^T – wejście obrócone w wiersz $(1, d_{in})$; $\delta \cdot x^T$ – kolumna razy wiersz, macierz (d_{out}, d_{in}), tego samego kształtu co W; $W^T\delta$ – kolumna $(d_{in}, 1)$, po jednej liczbie na każde wejście. (Tu δ piszemy jako kolumnę, tak jak na wykładzie. W zapisie z listy, gdzie gradient jest wierszem, to samo wygląda jak $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} W$.)

6.1. Skąd $\frac{\partial L}{\partial W} = \delta x^T$

1. rozpisuję jeden neuron: $z_i = W_{i1}x_1 + W_{i2}x_2 + \cdots + b_i$
2. waga W_{ij} występuje tylko w z_i (tylko neuron i jej używa), więc ma jedną drogę: $W_{ij} \to z_i \to L$
3. ogniwo $W_{ij} \to z_i$: w sumie stoi $W_{ij}x_j$, a reszta to stałe, więc $\frac{\partial z_i}{\partial W_{ij}} = x_j$
4. ogniwo $z_i \to L$: to δ_i, dostaliśmy je z góry
5. łańcuch: $\frac{\partial L}{\partial W_{ij}} = \delta_i \cdot x_j$
6. wszystkie kratki naraz: w wierszu i, kolumnie j stoi $\delta_i x_j$, a to jest dokładnie kolumna δ razy wiersz x^T (rozdz. B.5)

6.2. Skąd $\frac{\partial L}{\partial b} = \delta$

1. b_i występuje tylko w z_i i stoi tam „samo”, z mnożnikiem 1: $\frac{\partial z_i}{\partial b_i} = 1$
2. łańcuch: $\frac{\partial L}{\partial b_i} = \delta_i \cdot 1 = \delta_i$

6.3. Skąd $\frac{\partial L}{\partial x} = W^T\delta$

1. wejście x_j występuje w **każdym** neuronie: w z_1 z wagą W_{1j}, w z_2 z wagą W_{2j} itd. Kilka dróg, więc suma (rozdz. 2.4)
2. droga przez z_i: $\frac{\partial z_i}{\partial x_j} = W_{ij}$, razy δ_i
3. suma dróg: $\frac{\partial L}{\partial x_j} = \sum_i W_{ij}\,\delta_i = W_{1j}\delta_1 + W_{2j}\delta_2 + \ldots$
4. $W_{1j}, W_{2j}, \ldots$ to kolumna j macierzy W, czyli wiersz j macierzy W^T. Więc cały wektor to $W^T\delta$

Po ludzku: gradient wagi = (błąd neuronu, do którego waga prowadzi) × (wartość, która tą wagą wchodzi). Jeśli wejście było 0, waga nie miała wpływu i się nie zmienia. Błąd dla wejścia = suma błędów neuronów, ważona tym, jak mocno każdy neuron to wejście „słyszał”.

Na liczbach: $x = \begin{bmatrix}1\\2\end{bmatrix}$, $W = \begin{bmatrix}1 & 0\\2 & -1\end{bmatrix}$, $b = \begin{bmatrix}0\\1\end{bmatrix}$, z góry $\delta = \begin{bmatrix}0{,}5\\-1\end{bmatrix}$.

1. $\frac{\partial L}{\partial W} = \begin{bmatrix}0{,}5\\-1\end{bmatrix} \cdot [1\ \ 2] = \begin{bmatrix}0{,}5\cdot 1 & 0{,}5\cdot 2\\-1\cdot 1 & -1\cdot 2\end{bmatrix} = \begin{bmatrix}0{,}5 & 1\\-1 & -2\end{bmatrix}$; kształt 2×2, jak W ✓

2. $\dfrac{\partial L}{\partial b} = \begin{bmatrix} 0{,}5 \\ -1 \end{bmatrix}$
3. $W^T = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$; $\dfrac{\partial L}{\partial x_1} = 1 \cdot 0{,}5 + 2 \cdot (-1) = -1{,}5$; $\dfrac{\partial L}{\partial x_2} = 0 \cdot 0{,}5 + (-1) \cdot (-1) = 1$
4. $\dfrac{\partial L}{\partial x} = \begin{bmatrix} -1{,}5 \\ 1 \end{bmatrix}$

6.4. Wielu pacjentów naraz

Koszt to średnia po N pacjentach, więc gradient wag to **suma** gradientów policzonych dla każdego pacjenta osobno (a $\frac{1}{N}$ siedzi już w δ, rozdz. 7). W kodzie robi to jedno mnożenie `dW = dY.T @ X`.

Sprawdź się 6: $x = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$, $W = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$, z góry $\delta = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Policz $\frac{\partial L}{\partial W}$, $\frac{\partial L}{\partial b}$, $\frac{\partial L}{\partial x}$.

7. Wyjście sieci: sigmoida + entropia krzyżowa

> *„Zadaniem na to laboratorium jest zbudowanie modelu wielowarstwowego sieci neuronowej z funkcją aktywacji ReLU i funkcją kosztu taką, jak dla regresji logistycznej.”*

7.1. Koszt

$$L = -\frac{1}{N}\sum_{n=1}^{N}\Big[y_n \ln p_n + (1-y_n)\ln(1-p_n)\Big]$$

Co oznacza każdy symbol: N – liczba pacjentów; y_n – prawdziwa etykieta pacjenta n (1 chory, 0 zdrowy); p_n – prawdopodobieństwo choroby, które dała sieć; $\ln$ – logarytm naturalny; $\frac{1}{N}\sum$ – średnia.

Po ludzku: dla chorego ($y=1$) zostaje tylko $-\ln p$: kara mała, gdy p blisko 1, ogromna, gdy p blisko 0. Dla zdrowego ($y=0$) zostaje $-\ln(1-p)$: kara rośnie, gdy p rośnie. Model pewny i w błędzie płaci najwięcej.

Na liczbach:

1. chory, $p = 0{,}9$: $L = -\ln 0{,}9 = 0{,}105$ (dobrze, mała kara)
2. chory, $p = 0{,}1$: $L = -\ln 0{,}1 = 2{,}303$ (źle, duża kara)
3. zdrowy, $p = 0{,}731$: $L = -\ln(1-0{,}731) = -\ln 0{,}269 = 1{,}313$
4. każdy pacjent, $p = 0{,}5$: $L = -\ln 0{,}5 = \ln 2 = 0{,}693$ – **tyle kosztuje sieć, która nic nie wie**

7.2. Skąd $\delta = \frac{\partial L}{\partial z} = p - y$ (jeden pacjent)

Łańcuch: $z \to p \to L$, gdzie $p = \sigma(z)$, $L = -[y\ln p + (1-y)\ln(1-p)]$.

1. ogniwo $p \to L$: pochodna $\ln p$ to $\frac{1}{p}$, pochodna $\ln(1-p)$ to $\frac{-1}{1-p}$. Więc $\frac{\partial L}{\partial p} = -\left[\frac{y}{p} - \frac{1-y}{1-p}\right] = -\frac{y}{p} + \frac{1-y}{1-p}$
2. ogniwo $z \to p$: $\frac{\partial p}{\partial z} = p(1-p)$ (rozdz. C.4)
3. mnożę: $\left(-\frac{y}{p} + \frac{1-y}{1-p}\right)\cdot p(1-p) = -y(1-p) + (1-y)\,p$ (w pierwszym skraca się p, w drugim $1-p$)
4. wymnażam nawiasy: $-y + yp + p - yp = p - y$

$$\delta^{\text{wyj}} = \frac{\partial L}{\partial z} = p - y \qquad (\text{dla } N \text{ pacjentów: } \tfrac{p_n - y_n}{N} \text{ dla każdego})$$

Po ludzku: błąd wyjścia to po prostu „ile sieć powiedziała minus ile powinna”. $p = 0{,}731$ dla zdrowego: błąd $+0{,}731$ (za dużo). $p = 0{,}731$ dla chorego: błąd $-0{,}269$ (za mało).

Dlatego w kodzie sigmoida i koszt są jednym modułem (`SigmoidBCE`): liczone osobno wymagałyby dzielenia przez $p(1-p)$, które przy $p \approx 0$ lub 1 daje dzielenie przez prawie zero.

8. Cała sieć na liczbach: forward, backward, krok

Dane: sieć 2 → 2 → 1. Pacjent $x = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, **zdrowy**: $y = 0$. Wagi: $W^1 = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $b^1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $W^2 = [1{,}5 \;\; 2]$, $b^2 = -0{,}5$. Współczynnik uczenia $\alpha = 0{,}1$.

8.1. Forward (w kolejności operacji)

moduł	rachunek	wynik (zapamiętany)
Linear 1	$z_1^1 = 2 \cdot 1 + 1 \cdot (-1) + 0$; $z_2^1 = (-1) \cdot 1 + 3 \cdot (-1) + 0$	$z^1 = [1,\ -4]$
ReLU	$[\max(0,1),\ \max(0,-4)]$	$a^1 = [1,\ 0]$
Linear 2	$1{,}5 \cdot 1 + 2 \cdot 0 - 0{,}5$	$z^2 = 1$
sigmoida	$\frac{1}{1+e^{-1}} = \frac{1}{1{,}368}$	$p = 0{,}731$
koszt	$y = 0$, więc $-\ln(1 - 0{,}731) = -\ln 0{,}269$	$L = 1{,}313$

8.2. Backward (w kolejności odwrotnej)

1. **koszt + sigmoida**: $\delta^2 = p - y = 0{,}731 - 0 = 0{,}731$
2. **Linear 2, wagi**: $\frac{\partial L}{\partial W^2} = \delta^2 \cdot (a^1)^T = 0{,}731 \cdot [1,\ 0] = [0{,}731,\ 0]$; $\frac{\partial L}{\partial b^2} = 0{,}731$
3. **Linear 2, w dół**: $\frac{\partial L}{\partial a^1} = (W^2)^T \delta^2 = \begin{bmatrix} 1{,}5 \\ 2 \end{bmatrix} \cdot 0{,}731 = \begin{bmatrix} 1{,}097 \\ 1{,}462 \end{bmatrix}$
4. **ReLU**: maska $[z^1 > 0] = [1 > 0,\ -4 > 0] = [1,\ 0]$; $\delta^1 = [1{,}097,\ 1{,}462] \odot [1,\ 0] = [1{,}097,\ 0]$
5. **Linear 1, wagi**: $\frac{\partial L}{\partial W^1} = \delta^1 x^T = \begin{bmatrix} 1{,}097 \\ 0 \end{bmatrix} \cdot [1 \;\; -1] = \begin{bmatrix} 1{,}097 & -1{,}097 \\ 0 & 0 \end{bmatrix}$; $\frac{\partial L}{\partial b^1} = [1{,}097,\ 0]$

8.3. Krok uczenia: $W \leftarrow W - \alpha \frac{\partial L}{\partial W}$, $\alpha = 0{,}1$

1. $W^2 = [1{,}5 - 0{,}1 \cdot 0{,}731,\ \ 2 - 0{,}1 \cdot 0] = [1{,}427,\ 2]$
2. $b^2 = -0{,}5 - 0{,}1 \cdot 0{,}731 = -0{,}573$
3. $W^1 = \begin{bmatrix} 2 - 0{,}1 \cdot 1{,}097 & 1 - 0{,}1 \cdot (-1{,}097) \\ -1 - 0 & 3 - 0 \end{bmatrix} = \begin{bmatrix} 1{,}890 & 1{,}110 \\ -1 & 3 \end{bmatrix}$
4. $b^1 = [0 - 0{,}1 \cdot 1{,}097,\ 0 - 0] = [-0{,}110,\ 0]$

8.4. Sprawdzenie: czy koszt spadł?

Na liczbach: forward z nowymi wagami, ten sam pacjent $x = [1,\ -1]$:

1. $z_1^1 = 1{,}890 \cdot 1 + 1{,}110 \cdot (-1) - 0{,}110 = 0{,}671$; $z_2^1 = -4$ (bez zmian)
2. $a^1 = [0{,}671,\ 0]$
3. $z^2 = 1{,}427 \cdot 0{,}671 + 2 \cdot 0 - 0{,}573 = 0{,}384$
4. $p = \sigma(0{,}384) = 0{,}595$ (było 0,731)
5. $L = -\ln(1 - 0{,}595) = 0{,}904$ (było 1,313) – koszt spadł ✓
6. kontrola numeryczna: $\frac{L(W_{11}^1 + 10^{-6}) - L(W_{11}^1 - 10^{-6})}{2 \cdot 10^{-6}} = 1{,}0966$ – to samo co z backward ✓

Po ludzku: neuron 2 warstwy ukrytej był wyłączony ($z_2^1 = -4$), więc ani jego wagi, ani waga od niego do wyjścia się nie zmieniły. Poprawiał się tylko neuron 1, bo to on „przekonał" sieć, że zdrowy pacjent jest chory.

8.5. Ten sam algorytm ogólnie (dla dowolnej liczby warstw)

W przód, dla warstw $l = 1, \ldots, K$ (zaczynając od $a^0 = x$): $\quad z^l = W^l a^{l-1} + b^l, \quad a^l = f(z^l)$
Błąd na wyjściu: $\quad \delta^K = p - y$
Błąd wstecz, dla $l = K-1, \ldots, 1$: $\quad \delta^l = \left((W^{l+1})^T \delta^{l+1}\right) \odot f'(z^l)$
Gradienty: $\quad \frac{\partial L}{\partial W^l} = \delta^l (a^{l-1})^T, \quad \frac{\partial L}{\partial b^l} = \delta^l$
Krok: $\quad W^l \leftarrow W^l - \alpha \frac{\partial L}{\partial W^l}, \quad b^l \leftarrow b^l - \alpha \frac{\partial L}{\partial b^l}$

Co oznacza każdy symbol: l – numer warstwy; K – ostatnia warstwa; $a^0 = x$ – cechy pacjenta; $f'(z^l)$ – pochodna aktywacji (dla ReLU maska $[z^l > 0]$); δ^l – błąd warstwy l. Każda linijka to jedna rzecz z rozdz. 4, 6 i 7.

Uwaga na znaki: niektóre materiały piszą błąd jako $(y - p)$ i krok z „$+\alpha$". To jest to samo: minus jest tylko przeniesiony w inne miejsce.

Sprawdź się 8: Te same wagi ($W^1 = \begin{bmatrix} 2 & 1 \\ -1 & 3 \end{bmatrix}$, $b^1 = [0, 0]$, $W^2 = [1{,}5 \;\; 2]$, $b^2 = -0{,}5$, $\alpha = 0{,}1$), ten sam $x = [1, \; -1]$, ale pacjent **chory**: $y = 1$. Zrób 8.1–8.4. Jaki znak ma δ^2, w którą stronę zmienią się wagi i czy koszt spadnie?

9. Zadanie z części 1: co badamy i co wyszło

> *„Zadaniem na to laboratorium jest zbudowanie modelu wielowarstwowego sieci neuronowej z funkcją aktywacji ReLU i funkcją kosztu taką, jak dla regresji logistycznej. Następnie należy przebadać jak model zachowuje się na (zbinaryzowanym) zbiorze heart disease dla: różnej wymiarowości warstwy ukrytej (np. równa liczbie cech, znacznie większa od liczby cech, znacznie mniejsza), danych znormalizowanych i nieznormalizowanych, różnej liczby warstw. Inicjalizujemy wagi z rozkładu Gaussa z odchyleniem 0.01 i współczynnik uczenia 0.01."*

9.1. Ustawienia

co	wartość
dane	Heart Disease (Cleveland): 303 pacjentów, 13 badań, po zakodowaniu 17 cech. „Zbinaryzowany" = etykieta 1 chory (139 osób, 46%), 0 zdrowy (164 osoby)
podział	181 treningowy (uczenie), 61 walidacyjny (wybór momentu zatrzymania), 61 testowy (ocena), w każdym ok. 46% chorych
sieć	Linear → ReLU → … → Linear (1 neuron) → sigmoida, koszt: entropia krzyżowa
wagi startowe	losowane z $N(0;\ 0{,}01)$, czyli liczby rzędu ±0,01; biasy 0
uczenie	$\alpha = 0{,}01$, gradient z całego zbioru treningowego (1 krok na epokę)

9.2. Dlaczego koszt startuje z 0,69 i długo tam stoi (plateau)

1. wagi rzędu 0,01, więc z^1 jest małe, a^1 jeszcze mniejsze, $z^2 \approx 0$
2. $p = \sigma(0) = 0{,}5$ dla każdego pacjenta, $L = -\ln 0{,}5 = 0{,}693$
3. gradient wagi = błąd × wejście tej wagi (rozdz. 6), a wejścia warstw ukrytych są bliskie 0, więc gradient też jest bliski 0 i kroki są malutkie
4. koszt stoi na 0,69 setki epok. To **plateau** (płaskowyż), a nie minimum: z dobrym startem (inicjalizacja He) sieć od razu z niego schodzi

9.3. Trzy eksperymenty: co wyszło w Twoim raporcie

eksperyment	wynik	dlaczego
sieć bazowa 17-17-1	plateau ok. 720 epok; najlepsza na walidacji w epoce 1783, F1 test 0,764; po 30 000 epok koszt treningowy 0,031, walidacyjny rośnie do 0,909	sieć z warstwą ukrytą może prawie zapamiętać 181 pacjentów (przeuczenie), więc zapamiętujemy wagi z najlepszej epoki (early stopping)
szerokość: 2, 4 (mniej niż 17 cech), 17 (równo), 68, 256 (dużo więcej)	plateau 1055 epok (2 neurony) → 428 (256); 2 neurony kończą na koszcie 0,18, szerokie na 0,014–0,018; F1 przy early stopping ok. 0,76 dla każdej szerokości	więcej neuronów = gradient wyjścia jest sumą po większej liczbie neuronów, więc szybciej rusza; więcej wag = łatwiej zapamiętać trening

normalizacja: surowe / min-max [0, 1] / standaryzacja	surowe: sieć utyka na koszcie 0,52, F1 0,67, a sama regresja logistyczna się rozbiega; min-max i standaryzacja działają, standaryzacja szybciej	cholesterol ok. 250 obok cech 0/1: jeden krok α jest za długi dla dużej cechy i za krótki dla małych (uwarunkowanie 5,5 mln vs 46)
liczba warstw ukrytych: 0, 1, 2, 3, 4 (po 17)	wyjście z plateau po ok. 22 / 765 / 7700 / 94 600 epokach; 4 warstwy nie ruszyły w 150 000 epok; F1 sieci, które ruszyły, 0,75–0,76	gradient maleje kilka do kilkudziesięciu razy na każdą warstwę w stronę wejścia (zanikający gradient, rozdz. 10.5)
dodatkowo: He dla 3 i 4 warstw	wyjście z plateau po 24 i 55 epokach, F1 0,772	winna była za mała inicjalizacja, a nie sama głębokość

Po ludzku: mała sieć = mało „pokręteł” = wolniej rusza i gorzej pasuje do treningu. Duża = zapamięta nawet szum, więc trzeba ją w porę zatrzymać. Brak normalizacji psuje kroki. Dużo warstw przy wagach 0,01 = sygnał i gradient gasną po drodze.

Sprawdź się 9: (a) Dlaczego warstwa z 256 neuronami może mieć dużo niższy koszt na treningu niż z 17, a takie samo F1 na teście? (b) Dlaczego głęboka sieć z wagami $N(0;\ 0{,}01)$ prawie się nie uczy?

10. Część 2: optymalizacja

> *„Uczenie sieci z 2 lub więcej warstwami to problem optymalizacji niewypukłej, więc naszą pierwszą intuicją mogłoby być, że problemem są minima lokalne. To nie do końca prawda. (...) głównym problemem w optymalizacji sieci głębokich są przede wszystkim punkty siodłowe i płaskowyże. Innymi słowy, są to miejsca w których gradient zeruje się lub prawie się zeruje.”*

10.1. Wypukły i niewypukły koszt, punkt siodłowy

- **Wypukły** (jeden neuron z sigmoidą): kształt miski, jedno minimum, spadek gradientu zawsze do niego dojdzie.
- **Niewypukły** (sieć z warstwami ukrytymi): są miejsca, gdzie gradient ≈ 0, choć to nie jest minimum.
- **Punkt siodłowy**: w jednym kierunku dolina, w drugim górka (jak siodło konia). Gradient = 0.
- **Płaskowyż**: duży płaski obszar, gradient prawie 0, krok prawie 0, uczenie stoi.

Na liczbach: funkcja $f(x, y) = x^2 + \frac{y^4}{4} - \frac{y^2}{2}$.

1. gradient: $\frac{\partial f}{\partial x} = 2x$, $\frac{\partial f}{\partial y} = y^3 - y$
2. w punkcie (0, 0) oba są równe 0, ale: wzdłuż x funkcja x^2 to dolina, a wzdłuż y funkcja $\frac{y^4}{4} - \frac{y^2}{2}$ blisko zera to górka ($-\frac{y^2}{2}$ przeważa). To siodło. Minima są w $(0, \pm 1)$, $f = -0{,}25$
3. start $(1;\ 0{,}0001)$, $\alpha = 0{,}1$. Krok w y: $y \leftarrow y - 0{,}1(y^3 - y) \approx y + 0{,}1y = 1{,}1y$, więc y rośnie tylko o 10% na krok
4. żeby z 0,0001 dojść do ok. 0,5, trzeba $1{,}1^k = 5000$, czyli $k = \frac{\ln 5000}{\ln 1{,}1} \approx 89$ kroków. W Twoim raporcie: prosty gradient uciekł po **91** krokach, ADAM po **6**

10.2. Tryby uczenia: pełny zbiór, paczki, pojedyncze przykłady

Gradient liczony na paczce B pacjentów: $g = \frac{1}{|B|} \sum_{n \in B} \frac{\partial L_n}{\partial w}$

Co oznacza każdy symbol: B – zbiór pacjentów w paczce; $|B|$ – ich liczba; L_n – koszt jednego pacjenta n; g – średni gradient z paczki, nim robimy krok.

tryb	$\|B\|$	kroków na epokę (181 pacjentów)	cechy
pełny zbiór (batch GD)	181	1	dokładny gradient, gładki koszt, ale w płaskim miejscu stoi
paczki (mini-batch)	32	6 ($181/32 = 5{,}7$: 5 paczek po 32 i 1 po 21)	trochę szumu, dużo kroków, szybkie mnożenie macierzy
pojedyncze przykłady (SGD)	1	181	bardzo zaszumiony gradient, koszt skacze, wolne obliczenia

Na liczbach: 3 pacjentów, gradienty jednej wagi: $0{,}6$, $-0{,}2$, $0{,}5$.

1. pełny zbiór: $g = \frac{0{,}6 - 0{,}2 + 0{,}5}{3} = 0{,}3$, jeden krok

2. pojedyncze: trzy kroki z $g = 0{,}6$, potem $-0{,}2$, potem $0{,}5$. Raz w jedną stronę, raz w drugą – to jest szum
3. w punkcie siodłowym pełny gradient wynosi dokładnie 0, a gradienty pojedynczych pacjentów zwykle nie, więc szum może „wypchnąć" model z płaskiego miejsca

Twój wynik: koszt 0,3 po 398 / 66 / 4 epokach (pełny / paczki / pojedyncze), ale liczone w krokach podobnie (398–724). 1000 epok trwało ok. 0,3 / 0,8 / 15 s. F1 przy early stopping 0,76–0,78 we wszystkich trybach.

10.3. Momentum: średnia krocząca gradientów

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

Co oznacza każdy symbol: t – numer kroku; g_t – gradient w kroku t; m_t – pamięć kierunku (średnia ostatnich gradientów); $\beta_1 = 0{,}9$ – ile pamięci zostaje z poprzedniego kroku (90%), a $1 - \beta_1 = 0{,}1$ – ile wchodzi z nowego gradientu (10%); $m_0 = 0$.

Na liczbach: gradient stale $g = 1$, $\beta_1 = 0{,}9$:

1. $m_1 = 0{,}9 \cdot 0 + 0{,}1 \cdot 1 = 0{,}1$
2. $m_2 = 0{,}9 \cdot 0{,}1 + 0{,}1 \cdot 1 = 0{,}19$
3. $m_3 = 0{,}9 \cdot 0{,}19 + 0{,}1 = 0{,}271$, potem $0{,}344$, $0{,}410$ … i powoli do 1

Po ludzku: kulka toczy się z rozpędu. Gdy w płaskim miejscu gradient na chwilę spada do 0, m dalej jest duże, więc model „przejeżdża" przez siodło. Problem: na początku m jest za małe (0,1 zamiast 1), bo zaczyna od 0. Naprawia to korekcja biasu (10.4).

10.4. ADAM = momentum + skalowanie

„W tym drugim, skalowanie patrzy na historię „prędkości" i wymnaża obecny krok przez jej odwrotność, tak aby kierować się zasadą – jeśli ostatnie kilka kroków było zbyt wolne, to teraz chcemy przyspieszyć."

1. $m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$ — średnia gradientów (kierunek)
2. $v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$ — średnia kwadratów gradientów (wielkość)
3. $\hat{m}_t = \dfrac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \dfrac{v_t}{1 - \beta_2^t}$ — korekcja biasu
4. $w_t = w_{t-1} - \alpha \dfrac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$ — krok

Co oznacza każdy symbol: wszystko liczone **dla każdej wagi osobno**; g_t – gradient tej wagi w kroku t; g_t^2 – jego kwadrat (zawsze dodatni, mierzy wielkość bez znaku); $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ – ile pamięci zostaje; β_1^t – tu to **jest** potęga: $0{,}9^1 = 0{,}9$, $0{,}9^2 = 0{,}81$; $\sqrt{\hat{v}_t}$ – typowa wielkość gradientu; $\varepsilon = 10^{-8}$ – żeby nie dzielić przez 0; $m_0 = v_0 = 0$.

Po ludzku: krok to $\alpha \cdot \frac{\text{średni gradient}}{\text{typowa wielkość gradientu}}$. Jeśli gradient jest stale taki sam, ten ułamek to ok. 1 (albo −1), więc krok ma długość ok. α **niezależnie od tego, czy gradient to 1000 czy 0,000001**. Na płaskowyżu gradient jest malutki, ale krok nie.

Na liczbach, krok 1: $\alpha = 0{,}01$, dwie wagi: waga A ma gradient $g = 0{,}5$, waga B ma gradient $g = 0{,}01$ (50 razy mniejszy).

krok	**waga A ($g = 0{,}5$)**	**waga B ($g = 0{,}01$)**
$m_1 = 0{,}1 \cdot g$	$0{,}1 \cdot 0{,}5 = 0{,}05$	$0{,}1 \cdot 0{,}01 = 0{,}001$
$v_1 = 0{,}001 \cdot g^2$	$0{,}001 \cdot 0{,}25 = 0{,}00025$	$0{,}001 \cdot 0{,}0001 = 10^{-7}$
$\hat{m}_1 = m_1/(1-0{,}9)$	$0{,}05/0{,}1 = 0{,}5$	$0{,}001/0{,}1 = 0{,}01$
$\hat{v}_1 = v_1/(1-0{,}999)$	$0{,}00025/0{,}001 = 0{,}25$	$10^{-7}/0{,}001 = 0{,}0001$
$\sqrt{\hat{v}_1}$	$0{,}5$	$0{,}01$
krok $\alpha\hat{m}_1/\sqrt{\hat{v}_1}$	$0{,}01 \cdot 0{,}5/0{,}5 = 0{,}01$	$0{,}01 \cdot 0{,}01/0{,}01 = 0{,}01$
dla porównania: prosty gradient αg	$0{,}01 \cdot 0{,}5 = 0{,}005$	$0{,}01 \cdot 0{,}01 = 0{,}0001$

Obie wagi dostają krok 0,01, choć ich gradienty różnią się 50 razy. Prosty gradient zrobiłby dla wagi B krok 50 razy krótszy.

Na liczbach, krok 2: waga A, teraz $g_2 = 0{,}3$ (było $m_1 = 0{,}05$, $v_1 = 0{,}00025$):

1. $m_2 = 0{,}9 \cdot 0{,}05 + 0{,}1 \cdot 0{,}3 = 0{,}045 + 0{,}03 = 0{,}075$
2. $v_2 = 0{,}999 \cdot 0{,}00025 + 0{,}001 \cdot 0{,}09 = 0{,}00024975 + 0{,}00009 = 0{,}00033975$
3. $\hat{m}_2 = 0{,}075/(1-0{,}9^2) = 0{,}075/0{,}19 = 0{,}395$
4. $\hat{v}_2 = 0{,}00033975/(1-0{,}999^2) = 0{,}00033975/0{,}001999 = 0{,}170$, $\sqrt{0{,}170} = 0{,}412$
5. krok $= 0{,}01 \cdot 0{,}395/0{,}412 = 0{,}0096$ – dalej ok. α, bo gradient był podobny

Po co korekcja biasu: bez niej w kroku 1 mielibyśmy $m_1 = 0{,}1 \cdot g$, czyli 10 razy za mało, a $v_1 = 0{,}001 \cdot g^2$, czyli 1000 razy za mało. Dzielenie przez $1-0{,}9^1 = 0{,}1$ i $1-0{,}999^1 = 0{,}001$ dokładnie to naprawia. Dla dużego t: $0{,}9^t \to 0$, mianownik $\to 1$ i korekcja przestaje mieć znaczenie.

Twój wynik: start He: koszt 0,3 po 398 epokach (prosty gradient) vs 10–63 (ADAM). Start $N(0;\ 0{,}01)$: prosty gradient nie ruszył przez 3000 epok, ADAM wyszedł z plateau po 17–90 epokach. F1 podobne (0,76–0,77), ADAM tylko szybszy.

10.5. Zanikające gradienty

> *„Zarówno kiedy przechodzimy po warstwach w przód, jak i kiedy propagujemy w tył, dokonujemy wielokrotnego wymnażania macierzy. Jeżeli ta operacja zmniejsza wariancje aktywacji i jest wykonywana wielokrotnie, to dla pewnej liczby operacji (pewnej głębokości sieci) nasze gradienty dosłownie zanikną w początkowych warstwach.”*

Jak bardzo jedna warstwa zmienia „rozmiar” sygnału? Liczę wariancję (kwadrat typowej wielkości) jednego neuronu:

1. $z = W_1a_1 + W_2a_2 + \cdots + W_{d_{in}}a_{d_{in}}$: suma d_{in} składników
2. wagi są losowe i niezależne, średnia 0, więc wariancje składników się dodają: $\mathrm{Var}(z) = d_{in} \cdot \mathrm{Var}(W_ja_j)$

3. dla wagi o średniej 0: $\mathrm{Var}(W_j a_j) = \sigma_w^2 \cdot E[a_j^2]$, gdzie σ_w to odchylenie wag
4. ReLU zeruje ujemną połowę z z poprzedniej warstwy, więc $E[a^2] = \frac{1}{2}\mathrm{Var}(z_{\text{poprz}})$
5. razem: $\mathrm{Var}(z) = \frac{d_{in}\,\sigma_w^2}{2} \cdot \mathrm{Var}(z_{\text{poprz}})$

$$\text{mnożnik wariancji na warstwę} = \frac{d_{in}\,\sigma_w^2}{2}, \qquad \text{mnożnik wielkości (odchylenia)} = \sqrt{\frac{d_{in}\,\sigma_w^2}{2}}$$

Na liczbach: $d_{in} = 17$ (17 cech, warstwy po 17 neuronów), 5 warstw:

wagi	mnożnik wariancji $\frac{17\,\sigma_w^2}{2}$	odchylenie: razy na warstwę	wariancja po 5 warstwach	skutek (Twój raport)
$N(0;\ 0{,}01)$	$\frac{17 \cdot 0{,}0001}{2} = 0{,}00085$	$\times 0{,}029$ (maleje ok. 34 razy)	$0{,}00085^5 = 4 \cdot 10^{-16}$	gradient ok. 10^{-8}, sieć stoi na 0,69
$N(0;\ 1)$	$\frac{17 \cdot 1}{2} = 8{,}5$	$\times 2{,}9$	$8{,}5^5 = 44\,371$	koszt startowy 12, F1 test 0,50
He: $N\left(0;\ \sqrt{2/17}\right) = N(0;\ 0{,}343)$	$\frac{17 \cdot 0{,}1176}{2} = 1$	$\times 1$	1	stabilnie, F1 test 0,77
$N(0;\ 10)$, „zepsuta”	$\frac{17 \cdot 100}{2} = 850$	$\times 29$	$4 \cdot 10^{14}$	NaN po 1–2 epokach albo martwe ReLU

To samo mnożenie zachodzi wstecz, więc gradient pierwszych warstw maleje (albo rośnie) tak samo.

10.6. Inicjalizacja He

$$W \sim N\left(0;\ \sqrt{\frac{2}{d_{in}}}\right)$$

Co oznacza każdy symbol: $\sim$ – „losowane z”; $N(0;\ s)$ – rozkład normalny o średniej 0 i odchyleniu s; d_{in} – liczba wejść warstwy.

1. chcę mnożnik wariancji równy 1: $\frac{d_{in}\,\sigma_w^2}{2} = 1$
2. mnożę obie strony przez 2 i dzielę przez d_{in}: $\sigma_w^2 = \frac{2}{d_{in}}$
3. pierwiastek: $\sigma_w = \sqrt{\frac{2}{d_{in}}}$. Dla $d_{in} = 17$: $\sqrt{0{,}1176} = 0{,}343$

Po ludzku: He dobiera rozrzut wag tak, żeby każda warstwa ani nie zmniejszała, ani nie zwiększała sygnału. Wtedy gradient dociera do pierwszych warstw i sieć uczy się od pierwszej epoki. „2” bierze się stąd, że ReLU zeruje połowę wartości.

10.7. Zadanie z części 2

> *„Następnie, należy eksperymentalnie dokonać porównania: optymalizacji w trybie pełnego zbioru vs. pojedynczych przykładów vs. paczkowania; inicjalizacji z rozkładem o odchyleniu 1 vs. inicjalizacji He vs. inicjalizacji intencjonalnie „zepsutej” (np. bardzo duże odchylenie wag); optymalizacji przez prosty gradient vs. ADAM. Postaraj się, aby głębokość sieci i warunki stopu dla przebiegów uczenia były dobrane tak, aby problemy optymalizacji faktycznie dało się zauważyć w eksperymencie.”*

U Ciebie: sieć 17-17-17-17-17-17-1 (5 warstw ukrytych po 17 neuronów), celowo głęboka, bo przy jednej warstwie z He wszystkie metody działają prawie tak samo i problemów nie widać. Stała liczba epok dobrana do eksperymentu (1000 / 500 / 3000) i zapamiętywanie wag z najlepszej epoki na walidacji.

> **Sprawdź się 10:** (a) Policz odchylenie He dla warstwy z 50 wejściami. (b) W ADAM gradient wagi przez wiele kroków wynosi 0,001. Jaki będzie mniej więcej krok przy $\alpha = 0{,}01$? (c) Dlaczego SGD może wyjść z punktu siodłowego, a pełny gradient nie? (d) Policz mnożnik wariancji na warstwę dla $d_{in} = 17$ i $\sigma_w = 0{,}1$.

11. Ściąga wzorów

co	wzór	słowami
neuron / warstwa	$z = Wx + b,\ a = f(z)$	ważona suma + aktywacja
ReLU	$\max(0, z)$; pochodna 1 dla $z > 0$, 0 wpp.	ujemne → 0
sigmoida	$\sigma(z) = \frac{1}{1+e^{-z}}$; pochodna $\sigma(1-\sigma)$	liczba → (0, 1)
koszt	$L = -\frac{1}{N}\sum[y \ln p + (1-y)\ln(1-p)]$	start $\ln 2 = 0{,}693$
reguła łańcuchowa	$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$; kilka dróg → suma	mnożysz ogniwa
kształty	$\frac{\partial L}{\partial x}$: $(1, \dim_x)$; jakobian: $(\dim_y, \dim_x)$	wiersz = y, kolumna = x
backward aktywacji	$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \odot f'(z)$; ReLU: $\odot[z > 0]$	mnożenie element po elemencie
wyjście (sigmoida + CE)	$\delta = p - y$ (przez N dla średniej)	powiedziała minus powinna
warstwa liniowa	$\frac{\partial L}{\partial W} = \delta x^T,\ \frac{\partial L}{\partial b} = \delta,\ \frac{\partial L}{\partial x} = W^T\delta$	błąd × wejście
błąd wstecz	$\delta^l = ((W^{l+1})^T\delta^{l+1}) \odot f'(z^l)$	w dół przez wagi i maskę
krok	$W \leftarrow W - \alpha\frac{\partial L}{\partial W}$	pod górkę gradientu, więc minus
ADAM	$m = 0{,}9m + 0{,}1g;\ v = 0{,}999v + 0{,}001g^2;\ \hat{m} = \frac{m}{1-0{,}9^t};\ \hat{v} = \frac{v}{1-0{,}999^t};\ w \mathrel{-}= \alpha\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon}$	krok ≈ α niezależnie od skali gradientu
wariancja na warstwę	$\times\frac{d_{in}\sigma_w^2}{2}$	< 1 zanika, > 1 wybucha
He	$W \sim N\left(0;\ \sqrt{2/d_{in}}\right)$; dla 17 wejść 0,343	mnożnik = 1

12. Pytania na wyjściówkę

1. **Czym jest propagacja wsteczna?** Algorytmem liczenia gradientu kosztu po wszystkich wagach sieci regułą łańcuchową, od wyjścia do wejścia, tak żeby każdy wynik pośredni liczyć raz.
2. **Jakie są dwie fazy?** Forward: liczymy wyjście w kolejności operacji i zapamiętujemy wejścia. Backward: w odwrotnej kolejności liczymy $\frac{\partial L}{\partial x}$ i gradienty wag.
3. **Co przyjmuje i zwraca backward?** Przyjmuje $\frac{\partial L}{\partial y}$, zwraca $\frac{\partial L}{\partial x}$. Warstwa z wagami dodatkowo zapisuje $\frac{\partial L}{\partial W}$ i $\frac{\partial L}{\partial b}$.
4. **Dlaczego forward zapamiętuje wejścia?** Bo pochodne od nich zależą: dla x^2 potrzeba $2x$, dla warstwy liniowej x (do δx^T), dla ReLU maska $[z > 0]$.
5. **Po co funkcja aktywacji?** Bez niej $W^2(W^1 x) = (W^2 W^1)x$, czyli kilka warstw to dalej jedna warstwa liniowa.
6. **Wzór i pochodna ReLU, zalety, wady?** $\max(0, z)$, pochodna 1 albo 0. Prosta i nie nasyca się dla $z > 0$. Wada: neuron, który zawsze dostaje ujemne z, jest martwy (gradient 0).
7. **Dlaczego dla aktywacji $\odot$, a nie pełne mnożenie macierzy?** Jakobian funkcji element-wise jest diagonalny, a mnożenie przez macierz diagonalną to mnożenie element po elemencie.
8. **Ile wynosi δ na wyjściu dla sigmoidy i entropii krzyżowej?** $p - y$, bo $p(1-p)$ z pochodnej sigmoidy skraca się z mianownikami z pochodnej logarytmu.
9. **Gradient wag warstwy liniowej?** δx^T: błąd neuronu × wejście wagi. Kształt taki jak W.
10. **Jak błąd przechodzi do warstwy niżej?** $\delta^l = ((W^{l+1})^T \delta^{l+1}) \odot f'(z^l)$.
11. **Jak sprawdzić, czy backward jest dobry?** Gradient numeryczny $\frac{L(w+h) - L(w-h)}{2h}$ dla każdej wagi; u Ciebie zgodność ok. 10^{-10}.
12. **Dlaczego koszt startuje z 0,693?** Wagi ≈ 0 → $z \approx 0$ → $p = 0{,}5$ → $L = -\ln 0{,}5 = 0{,}693$.
13. **Czym jest plateau i punkt siodłowy?** Miejscami, gdzie gradient ≈ 0, choć to nie minimum. Prosty gradient tam stoi.
14. **Pełny zbiór vs paczki vs pojedyncze?** Krok po całym zbiorze / po paczce (np. 32) / po każdym przykładzie. Mniejsza paczka = więcej kroków i więcej szumu.
15. **Jak działa ADAM?** Średnia gradientów m (momentum) podzielona przez pierwiastek średniej kwadratów v (skalowanie), z korekcją biasu. Krok każdej wagi ≈ α niezależnie od wielkości gradientu.
16. **Po co korekcja biasu?** m i v startują od 0, więc na początku są zaniżone (10 i 1000 razy). Dzielenie przez $1 - \beta^t$ to wyrównuje.
17. **Co to zanikający gradient?** Każda warstwa mnoży sygnał (i gradient) przez $\sqrt{d_{in}\sigma_w^2/2}$. Przy wagach 0,01 i 17 wejściach to 0,03, więc po kilku warstwach gradient pierwszych warstw jest praktycznie 0.
18. **Inicjalizacja He?** $\sigma_w = \sqrt{2/d_{in}}$, mnożnik wariancji = 1, sygnał ma stałą wielkość w każdej warstwie.
19. **Czy sieć wielowarstwowa zawsze dojdzie do minimum globalnego?** Nie, bo koszt jest niewypukły. Ale główną przeszkodą w praktyce są siodła i płaskowyże, nie minima lokalne.

13. Rozwiązania „Sprawdź się”

B

(a) wiersz 1: $2 \cdot (-1) + 1 \cdot 2 = 0$; wiersz 2: $(-1)(-1) + 3 \cdot 2 = 7$. $Wx = \begin{bmatrix} 0 \\ 7 \end{bmatrix}$

(b) $(8, 17) \cdot (17, 1) = (8, 1)$

(c) $\begin{bmatrix} 1 \cdot 3 & 1 \cdot (-1) \\ 2 \cdot 3 & 2 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}$

C

(a) $f'(x) = 2x = 2 \cdot (-2) = -4$. Zmiana: $-4 \cdot 0{,}01 = -0{,}04$, czyli f zmaleje z 4 do ok. 3,96 (dokładnie $(-1{,}99)^2 = 3{,}9601$)

(b) $\dfrac{\partial f}{\partial x_1} = 2x_1 = 2$; $\dfrac{\partial f}{\partial x_2} = 4$

1

1. $z_1^1 = 2 \cdot (-1) + 1 \cdot 2 + 0 = 0$; $z_2^1 = (-1)(-1) + 3 \cdot 2 + 0 = 7$. $z^1 = [0,\ 7]$
2. $a^1 = [\max(0, 0),\ \max(0, 7)] = [0,\ 7]$
3. $z^2 = 1{,}5 \cdot 0 + 2 \cdot 7 - 0{,}5 = 13{,}5$
4. $p = \sigma(13{,}5) \approx 0{,}99999$: sieć jest prawie pewna, że pacjent jest chory

2

$z = 3 \cdot 1 = 3$, $L = 9$, $\dfrac{\partial L}{\partial z} = 2z = 6$, $\dfrac{\partial z}{\partial w} = 3$, $\dfrac{\partial L}{\partial w} = 6 \cdot 3 = 18$. Sprawdzenie: $L = 9w^2$, pochodna $18w = 18$ ✓

3

(a) x: $(17, 1)$; jakobian: $(8, 17)$; $\dfrac{\partial L}{\partial x}$: $(1, 17)$

(b) tabelka (wiersze y_1, y_2; kolumny x_1, x_2, x_3): y_1: $[1,\ 0,\ 2]$, y_2: $[0,\ 3,\ -1]$, z góry $[2,\ 5]$. kolumna x_1: $1 \cdot 2 + 0 \cdot 5 = 2$; kolumna x_2: $0 \cdot 2 + 3 \cdot 5 = 15$; kolumna x_3: $2 \cdot 2 + (-1) \cdot 5 = -1$. $\dfrac{\partial L}{\partial x} = [2,\ 15,\ -1]$

4

maska: $-1 > 0$ nie, $2 > 0$ tak, $0{,}5 > 0$ tak → $[0,\ 1,\ 1]$. $\dfrac{\partial L}{\partial z} = [3,\ 1,\ 4] \odot [0,\ 1,\ 1] = [0,\ 1,\ 4]$

5

(a) Bo pochodna zależy od wejścia ($2x$ dla x^2, x dla wag warstwy liniowej, maska dla ReLU), a w backward wejścia nie liczymy od nowa.

(b) Oddaje w dół $\dfrac{\partial L}{\partial x} = W^T \delta$. Zatrzymuje $\dfrac{\partial L}{\partial W}$ i $\dfrac{\partial L}{\partial b}$ do kroku uczenia.

6

1. $\dfrac{\partial L}{\partial W} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 3 & -1 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 6 & -2 \end{bmatrix}$
2. $\dfrac{\partial L}{\partial b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

3. $W^T = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$; $\dfrac{\partial L}{\partial x_1} = 2 \cdot 1 + 0 \cdot 2 = 2$; $\dfrac{\partial L}{\partial x_2} = 1 \cdot 1 + 1 \cdot 2 = 3$. $\dfrac{\partial L}{\partial x} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$

8 (pacjent chory, $y = 1$)

1. forward bez zmian: $z^1 = [1,\ -4]$, $a^1 = [1,\ 0]$, $z^2 = 1$, $p = 0{,}731$; koszt $L = -\ln 0{,}731 = 0{,}313$

2. $\delta^2 = 0{,}731 - 1 = -0{,}269$ – **ujemne**: sieć powiedziała za mało

3. $\dfrac{\partial L}{\partial W^2} = -0{,}269 \cdot [1,\ 0] = [-0{,}269,\ 0]$; $\dfrac{\partial L}{\partial b^2} = -0{,}269$

4. $\dfrac{\partial L}{\partial a^1} = [1{,}5 \cdot (-0{,}269),\ 2 \cdot (-0{,}269)] = [-0{,}403,\ -0{,}538]$; $\delta^1 = [-0{,}403,\ -0{,}538] \odot [1,\ 0] = [-0{,}403,\ 0]$

5. $\dfrac{\partial L}{\partial W^1} = \begin{bmatrix} -0{,}403 \\ 0 \end{bmatrix} [1 \quad -1] = \begin{bmatrix} -0{,}403 & 0{,}403 \\ 0 & 0 \end{bmatrix}$; $\dfrac{\partial L}{\partial b^1} = [-0{,}403,\ 0]$

6. krok (minus razy minus = plus): $W^2 = [1{,}5 + 0{,}027,\ 2] = [1{,}527,\ 2]$; $b^2 = -0{,}5 + 0{,}027 = -0{,}473$; $W^1_{1\cdot} = [2 + 0{,}040,\ 1 - 0{,}040] = [2{,}040,\ 0{,}960]$; $b^1 = [0{,}040,\ 0]$

7. sprawdzenie: $z^1_1 = 2{,}040 - 0{,}960 + 0{,}040 = 1{,}121$; $z^2 = 1{,}527 \cdot 1{,}121 - 0{,}473 = 1{,}239$; $p = \sigma(1{,}239) = 0{,}775$ (było 0,731); $L = -\ln 0{,}775 = 0{,}254$ (było 0,313) ✓

Wagi neuronu 1 i waga od niego do wyjścia zmieniły się w przeciwną stronę niż dla zdrowego: sieć ma być bardziej pewna choroby. Neuron 2 dalej wyłączony, bez zmian.

9

(a) 256 neuronów to ponad 4600 wag na 181 pacjentów: sieć zapamiętuje szczegóły i szum treningu (koszt 0,014), a to nie przenosi się na nowych pacjentów. Przy early stopping bierzemy wagi sprzed przeuczenia, więc F1 jest podobne (ok. 0,76).

(b) Każda warstwa zmniejsza sygnał ok. 34 razy ($\sqrt{17 \cdot 0{,}0001/2} = 0{,}029$), wstecz gradient tak samo. Po kilku warstwach $p = 0{,}5$ dla każdego, a gradient pierwszych warstw jest praktycznie 0, więc kroki są zerowe.

10

(a) $\sqrt{2/50} = \sqrt{0{,}04} = 0{,}2$

(b) $m \to 0{,}001$, $v \to 0{,}001^2 = 10^{-6}$, $\sqrt{v} = 0{,}001$; krok $\approx 0{,}01 \cdot \dfrac{0{,}001}{0{,}001} = 0{,}01$. ADAM „nie zauważa”, że gradient jest mały; prosty gradient zrobiłby krok $0{,}00001$

(c) Gradient z jednego przykładu jest zaszumiony i w siodle zwykle nie wynosi dokładnie 0, więc model się przesuwa. Pełny gradient w siodle wynosi dokładnie 0.

(d) $\dfrac{17 \cdot 0{,}01}{2} = 0{,}085$: sygnał maleje, odchylenie razy $\sqrt{0{,}085} = 0{,}29$ na warstwę